Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №2

По дисциплине «Методы обработки изображений»

Выполнил:

Студент группы P3306,

Михайлов Дмитрий Андреевич

Преподаватель:

Потемин Игорь Станиславович

Санкт-Петербург

2026 год

Содержание

Задание 3
Шаги алгоритма 4
Листинг программы 5
Параметры вычислительного эксперимента 14
Результаты вычислений 14
1. Аналитическое решение 15
2. Простое Монте-Карло 15
3. Монте-Карло со стратификацией 15
4. Выборка по значимости 16
5. Многократная выборка по значимости (MIS) 16
6. Монте-Карло с русской рулеткой 16
Анализ результатов вычислений 17
Вывод 18

Задание

Изучить базовые методы Монте-Карло, используемые для интегрирования функций на заданном интервале, и оценить точность интегрирования и основные преимущества и недостатки этих методов.

Задана функция $f(x) = x^2$. Вычислить интеграл данной функции на интервале $[2, 5]$. Интеграл вычисляется:

1. Аналитически.
2. Простым интегрированием методом Монте-Карло.
3. Интегрированием методом Монте-Карло со стратификацией. Использовать два варианта разбиения: с шагом 1 и с шагом 0.5.
4. Интегрированием методом Монте-Карло с выборкой по значимости. Использовать три варианта плотности вероятности $p_1(x) = x, p_2(x) = x^2, p_3(x) = x^3$.
5. Интегрированием методом Монте-Карло с многократной выборкой по значимости. Использовать две функции плотности вероятности $p(x) = x, p(x) = x^3$. В качестве весов использовать среднюю плотность вероятности (первый вариант расчета $w_1(x) = \frac{p_1(x)}{p_1(x)+p_2(x)}, w_2(x) = \frac{p_2(x)}{p_1(x)+p_2(x)}$) и средний квадрат двух плотностей вероятности (второй вариант расчета $w_1(x) = \frac{p_1^2(x)}{p_1^2(x)+p_2^2(x)}, w_2(x) = \frac{p_2^2(x)}{p_1^2(x)+p_2^2(x)}$).
6. Интегрированием методом Монте-Карло с использованием русской рулетки. Использовать три точки отсечки: $R_1 = 0.5, R_2 = 0.75, R_3 = 0.95$.

При вычислении использовать программный генератор псевдослучайных чисел. Использовать разное число выборок при расчете интеграла: $N_1 = 100, N_2 = 1000, N_3 = 10000, N_5 = 100000$.

Результаты представить в виде отчета, в который содержит таблицы значений истинного интеграла, интеграла, вычисленного каждым из методов Монте-Карло, погрешности вычислений и оценки погрешности $\Delta I = \frac{I_{true}}{\sqrt{N}}$.

Представит код программы. Язык программирования произвольный.

Шаги алгоритма

1. Задание исходных данных.
2. Аналитическое вычисление интеграла.
3. Вычисление интеграла методом простого Монте-Карло.
4. Вычисление интеграла методом Монте-Карло со стратификацией.
5. Вычисление интеграла методом Монте-Карло с выборкой по значимости.
6. Вычисление интеграла методом многократной выборки по значимости.
7. Вычисление интеграла методом Монте-Карло с использованием русской рулетки.
8. Вычисление погрешностей.
9. Вывод результатов в таблицы.
10. Сравнение методов.

Листинг программы

1. Задание исходных данных

На первом этапе в программе задаются границы интервала интегрирования, наборы чисел выборок, параметры стратификации, значения для метода русской рулетки и начальное значение генератора случайных чисел.

```
import math
import random
from typing import List, Tuple



A = 2.0
B = 5.0
N_VALUES = [100, 1000, 10000, 100000]
STRAT_STEPS = [1.0, 0.5]
RUSSIAN_R_VALUES = [0.5, 0.75, 0.95]
SEED = 42
```

Здесь:

- A и B — границы интегрирования;
- N_VALUES — список количеств случайных выборок;
- STRAT_STEPS — шаги разбиения интервала для стратификации;
- RUSSIAN_R_VALUES — значения точек отсечки для метода русской рулетки;
- SEED — начальное значение генератора псевдослучайных чисел.

2. Задание подынтегральной функции и аналитическое вычисление интеграла

Далее задается подынтегральная функция $f(x) = x^2$и функция для вычисления точного значения интеграла по аналитической формуле.

```
def f(x: float) -> float:
    return x * x


def true_integral(a: float, b: float) -> float:
    return (b ** 3 - a ** 3) / 3.0
```

```
I_TRUE = true_integral(A, B)
```

Функция f(x) возвращает значение x^2.
Функция true_integral(a, b) вычисляет точное значение интеграла:

$$\int_a^b x^2\,dx = \frac{b^3 - a^3}{3}$$

Для интервала $[2, 5]$получается значение:

$$I_{true} = 39$$

3. Вычисление интеграла методом простого Монте-Карло

Для простого метода Монте-Карло на интервале $[A, B]$генерируются равномерно распределенные случайные точки, для которых вычисляется среднее значение функции.

```
def simple_monte_carlo(n: int, rng: random.Random) -> float:
    s = 0.0
    for _ in range(n):
        x = rng.uniform(A, B)
        s += f(x)
    return (B - A) * s / n
```

В данной функции:

- генерируется n случайных точек на отрезке $[2, 5]$;
- для каждой точки вычисляется $f(x)$;
- итоговая оценка интеграла получается умножением среднего значения функции на длину интервала.

4. Вычисление интеграла методом Монте-Карло со стратификацией

В этом методе весь интервал разбивается на подынтервалы одинаковой длины. На каждом подынтервале интеграл оценивается отдельно, а затем результаты суммируются.

```
def stratified_monte_carlo(n: int, step: float, rng: random.Random) ->
float:
    strata = []
    left = A
    while left < B - 1e-12:
        right = min(left + step, B)
        strata.append((left, right))
```

```
        left = right

    m = len(strata)

    base = n // m
    rem = n % m
    counts = [base + (1 if i < rem else 0) for i in range(m)]

    estimate = 0.0
    for (l, r), cnt in zip(strata, counts):
        if cnt == 0:
            continue
        local_sum = 0.0
        for _ in range(cnt):
            x = rng.uniform(l, r)
            local_sum += f(x)
        estimate += (r - l) * (local_sum / cnt)

    return estimate
```

Алгоритм работы функции:

1. формируется список подынтервалов длиной step;
2. общее число выборок n распределяется между всеми стратами;
3. внутри каждой страты генерируются случайные точки;
4. для каждой страты отдельно считается вклад в интеграл;
5. итоговое значение равно сумме вкладов всех подынтервалов.

В программе используются два значения шага стратификации: 1.0 и 0.5.

5. Вычисление интеграла методом Монте-Карло с выборкой по значимости

Для выборки по значимости в программе используются плотности вероятности, пропорциональные степеням x. Для этого сначала вычисляется нормировочная константа, затем сама плотность и способ генерации случайной точки.

```
def norm_const_power(k: int, a: float, b: float) -> float:
    return (k + 1) / (b ** (k + 1) - a ** (k + 1))



def pdf_power(x: float, k: int, a: float, b: float) -> float:
    c = norm_const_power(k, a, b)
    return c * (x ** k)



def sample_power_pdf(k: int, a: float, b: float, rng: random.Random) ->
float:
    u = rng.random()
    power = k + 1
    return (u * (b ** power - a ** power) + a ** power) ** (1.0 / power)
```

Функция norm_const_power вычисляет коэффициент нормировки для плотности вида $p(x) = cx^k$.
Функция pdf_power возвращает значение нормированной плотности.
Функция sample_power_pdf генерирует случайное значение x по данной плотности методом обратной функции распределения.

После этого реализуется сам метод выборки по значимости:

```
def importance_sampling(n: int, k: int, rng: random.Random) -> float:
    s = 0.0
    for _ in range(n):
        x = sample_power_pdf(k, A, B, rng)
        p = pdf_power(x, k, A, B)
        s += f(x) / p
    return s / n
```

Здесь:

- случайные точки генерируются неравномерно;
- для каждой точки вычисляется отношение $\frac{f(x)}{p(x)}$;
- итоговая оценка интеграла равна среднему значению этой величины. В программе используются три варианта плотности: $p(x) \sim x, p(x) \sim x^2, p(x) \sim x^3$.

6. Вычисление интеграла методом многократной выборки по значимости

Для многократной выборки по значимости используются две плотности вероятности и два варианта весовых коэффициентов.

Сначала задаются функции вычисления весов:

```
   def mis_weights_balance(x: float, p1: float, p2: float) -> Tuple[float, float]:
       denom = p1 + p2
       return p1 / denom, p2 / denom



   def mis_weights_power2(x: float, p1: float, p2: float) -> Tuple[float, float]:
       p1_2 = p1 * p1
       p2_2 = p2 * p2
       denom = p1_2 + p2_2
       return p1_2 / denom, p2_2 / denom
```

В первом случае используются веса:

$$w_1(x) = \frac{p_1(x)}{p_1(x) + p_2(x)}, w_2(x) = \frac{p_2(x)}{p_1(x) + p_2(x)}$$

Во втором случае:

$$w_1(x) = \frac{p_1^2(x)}{p_1^2(x) + p_2^2(x)}, w_2(x) = \frac{p_2^2(x)}{p_1^2(x) + p_2^2(x)}$$

После этого реализуется основной алгоритм MIS:

```
   def multiple_importance_sampling(n: int, weight_mode: str, rng: random.Random) -> float:
       n1 = n // 2
       n2 = n - n1

       if weight_mode == "balance":
           weight_func = mis_weights_balance
       elif weight_mode == "power2":
           weight_func = mis_weights_power2
       else:
           raise ValueError("Неизвестный режим весов")

       part1 = 0.0
       part2 = 0.0
```

```
    for _ in range(n1):
        x = sample_power_pdf(1, A, B, rng)
        p1 = pdf_power(x, 1, A, B)
        p2 = pdf_power(x, 3, A, B)
        w1, _ = weight_func(x, p1, p2)
        part1 += w1 * f(x) / p1

    for _ in range(n2):
        x = sample_power_pdf(3, A, B, rng)
        p1 = pdf_power(x, 1, A, B)
        p2 = pdf_power(x, 3, A, B)
        _, w2 = weight_func(x, p1, p2)
        part2 += w2 * f(x) / p2

    result = 0.0
    if n1 > 0:
        result += part1 / n1
    if n2 > 0:
        result += part2 / n2

    return result
```

В этой функции:

- половина выборок строится по плотности $p_1(x) \sim x$;
- вторая половина — по плотности $p_2(x) \sim x^3$;
- для каждой выборки учитывается соответствующий вес;
- итоговая оценка интеграла получается объединением вкладов от двух распределений.

7. Вычисление интеграла методом Монте-Карло с использованием русской рулетки

Метод русской рулетки реализуется с помощью случайного отбора: вклад точки либо сохраняется, либо отбрасывается, а при сохранении корректируется на вероятность прохождения.

```
def russian_roulette_monte_carlo(n: int, r_cutoff: float, rng: random.Random) -> float:
    q = 1.0 - r_cutoff
    if q <= 0.0:
        raise ValueError("R должно быть меньше 1")

    s = 0.0
    for _ in range(n):
        x = rng.uniform(A, B)
        if rng.random() < q:
            s += f(x) / q
        else:
            s += 0.0

    return (B - A) * s / n
```

Здесь:

- q = 1 - R — вероятность того, что вклад точки будет учтен;
- если случайное число меньше q, значение функции включается в сумму с поправочным множителем 1/q;
- если условие не выполняется, вклад считается равным нулю.

В программе рассматриваются три точки отсечки: 0.5, 0.75, 0.95.

8. Вычисление абсолютной ошибки и оценки погрешности

После вычисления приближенного значения интеграла определяется абсолютная ошибка и оценка погрешности, указанная в задании.

```
def error_abs(estimate: float) -> float:
    return abs(I_TRUE - estimate)



def delta_estimate(n: int) -> float:
    return I_TRUE / math.sqrt(n)
```

Функция error_abs вычисляет модуль разности между точным и приближенным значениями интеграла.
Функция delta_estimate вычисляет величину:

$$\Delta I = \frac{I_{true}}{\sqrt{N}}$$

которая используется в таблицах результатов.

9. Вывод результатов в таблицы

Для удобного представления результатов в программе используется отдельная функция печати таблиц.

```
def print_header(title: str) -> None:
    print("\n" + "=" * 110)
    print(title)
    print("=" * 110)



def print_table(title: str, rows: List[Tuple]) -> None:
    print_header(title)
    print(
        f"{'Параметр':<20}"
        f"{'N':>10}"
        f"{'I_true':>18}"
        f"{'I_MC':>18}"
        f"{'|Ошибка|':>18}"
        f"{'ΔI=I_true/sqrt(N)':>22}"
    )
    print("-" * 110)
    for param, n, i_true, i_mc, err, delta in rows:
        print(
            f"{str(param):<20}"
            f"{n:>10}"
            f"{i_true:>18.8f}"
            f"{i_mc:>18.8f}"
            f"{err:>18.8f}"
            f"{delta:>22.8f}"
        )
```

Данная функция выводит таблицу со следующими столбцами:

- параметр метода;
- число выборок;
- точное значение интеграла;
- вычисленное значение;
- абсолютная ошибка;
- оценка погрешности.

10.Основная функция программы

Основная функция main() последовательно вызывает все рассмотренные методы, собирает результаты и выводит их на экран.

```
def main() -> None:
    print(f"Функция: f(x) = x^2")
    print(f"Интервал: [{A}, {B}]")
    print(f"Истинное значение интеграла: {I_TRUE:.8f}")
    print(f"Используемые N: {N_VALUES}")
    print(f"Шаги стратификации: {STRAT_STEPS}")
    print(f"Пороги русской рулетки: {RUSSIAN_R_VALUES}")
    print(f"Seed генератора: {SEED}")

    print_header("АНАЛИТИЧЕСКОЕ РЕШЕНИЕ")
    print("∫[2,5] x^2 dx = (5^3 - 2^3) / 3 = 39.0")
```

Далее в функции формируются таблицы для каждого метода. Например, для простого Монте-Карло:

```
    rows = []
    for n in N_VALUES:
        rng = random.Random(SEED + 1000 + n)
        estimate = simple_monte_carlo(n, rng)
        rows.append((
            "-",
            n,
            I_TRUE,
            estimate,
            error_abs(estimate),
            delta_estimate(n),
        ))
```

```
        print_table("ПРОСТОЕ МОНТЕ-КАРЛО", rows)
```

Аналогично строятся таблицы для:

- стратифицированного метода;
- выборки по значимости;
- многократной выборки по значимости;
- метода русской рулетки.

Параметры вычислительного эксперимента

В программе использовались следующие исходные параметры:

- подынтегральная функция: $f(x) = x^2$;
- интервал интегрирования: $[2, 5]$;
- количество выборок: $N = 100, 1000, 10000, 100000$;
- шаги стратификации: $h = 1$и $h = 0.5$;
- плотности вероятности для выборки по значимости: $p(x) \sim x, p(x) \sim x^2, p(x) \sim x^3$;
- плотности вероятности для многократной выборки по значимости: $p_1(x) \sim x, p_2(x) \sim x^3$;
- точки отсечки для русской рулетки: $R = 0.5, 0.75, 0.95$;
- начальное значение генератора псевдослучайных чисел: SEED = 42.

Точное значение интеграла, вычисленное аналитически:

$$I_{true} = \int_2^5 x^2\, dx = 39.$$

Результаты вычислений

В ходе выполнения лабораторной работы было получено точное значение интеграла функции $f(x) = x^2$на интервале $[2, 5]$аналитическим способом:

$$I_{true} = \int_2^5 x^2\ dx = 39.$$

Далее были выполнены вычисления методом Монте-Карло и его модификациями при числе выборок

$$N = 100, 1000, 10000, 100000.$$

Для каждого метода определялись:

- точное значение интеграла I_{true};
- вычисленное значение I_{MC};
- абсолютная ошибка $| I_{true} - I_{MC} |$;
- оценка погрешности

$$\Delta I = \frac{I_{true}}{\sqrt{N}}$$

1. Аналитическое решение

Точное значение интеграла: I_true = 39.00.

2. Простое Монте-Карло

Параметр	N	I_true	I_MC	Ошибка	ΔI
-	100	39.00000000	35.62947805	3.37052195	3.90000000
-	1000	39.00000000	39.63194269	0.63194269	1.23328829
-	10000	39.00000000	38.69774967	0.30225033	0.39000000
-	100000	39.00000000	39.04491913	0.04491913	0.12332883

3. Монте-Карло со стратификацией

Шаг стратификации h = 1.0

Параметр	N	I_true	I_MC	Ошибка	ΔI
step = 1.0	100	39.00000000	39.39458775	0.39458775	3.90000000
step = 1.0	1000	39.00000000	38.77184777	0.22815223	1.23328829
step = 1.0	10000	39.00000000	39.06642659	0.06642659	0.39000000
step = 1.0	100000	39.00000000	39.00054100	0.00054100	0.12332883

Шаг стратификации h = 0.5

Параметр	N	I_true	I_MC	Ошибка	ΔI
step = 0.5	100	39.00000000	38.99316587	0.00683413	3.90000000
step = 0.5	1000	39.00000000	39.17003537	0.17003537	1.23328829
step = 0.5	10000	39.00000000	39.01455336	0.01455336	0.39000000
step = 0.5	100000	39.00000000	39.00644600	0.00644600	0.12332883

4. Выборка по значимости

Плотность p(x) ~ x

Параметр	N	I_true	I_MC	Ошибка	ΔI
p(x) ~ x	100	39.00000000	38.65391304	0.34608696	3.90000000
p(x) ~ x	1000	39.00000000	39.62030678	0.62030678	1.23328829
p(x) ~ x	10000	39.00000000	38.87343966	0.12656034	0.39000000
p(x) ~ x	100000	39.00000000	39.01883524	0.01883524	0.12332883

Плотность p(x) ~ x^2

Параметр	N	I_true	I_MC	Ошибка	ΔI
p(x) ~ x^2	100	39.00000000	39.00000000	0.00000000	3.90000000
p(x) ~ x^2	1000	39.00000000	39.00000000	0.00000000	1.23328829
p(x) ~ x^2	10000	39.00000000	39.00000000	0.00000000	0.39000000
p(x) ~ x^2	100000	39.00000000	39.00000000	0.00000000	0.12332883

Плотность p(x) ~ x^3

Параметр	N	I_true	I_MC	Ошибка	ΔI
p(x) ~ x^3	100	39.00000000	39.15782727	0.15782727	3.90000000
p(x) ~ x^3	1000	39.00000000	39.14598893	0.14598893	1.23328829
p(x) ~ x^3	10000	39.00000000	38.69496075	0.30503925	0.39000000
p(x) ~ x^3	100000	39.00000000	39.02868412	0.02868412	0.12332883

5. Многократная выборка по значимости (MIS)

Весовая схема balance

Параметр	N	I_true	I_MC	Ошибка	ΔI
balance	100	39.00000000	39.12851889	0.12851889	3.90000000
balance	1000	39.00000000	39.00994371	0.00994371	1.23328829
balance	10000	39.00000000	38.96655968	0.03344032	0.39000000
balance	100000	39.00000000	38.99895423	0.00104577	0.12332883

Весовая схема power2

Параметр	N	I_true	I_MC	Ошибка	ΔI
power2	100	39.00000000	39.42133690	0.42133690	3.90000000
power2	1000	39.00000000	38.63723221	0.36276779	1.23328829
power2	10000	39.00000000	39.05130994	0.05130994	0.39000000
power2	100000	39.00000000	38.95451330	0.04548670	0.12332883

6. Монте-Карло с русской рулеткой

Точка отсечки R = 0.5

Параметр	N	I_true	I_MC	Ошибка	ΔI
R = 0.5	100	39.00000000	52.45369724	13.45369724	3.90000000
R = 0.5	1000	39.00000000	36.92047464	2.07952536	1.23328829
R = 0.5	10000	39.00000000	38.80037496	0.19962504	0.39000000
R = 0.5	100000	39.00000000	39.00836372	0.00836372	0.12332883

Точка отсечки R = 0.75

Параметр	N	I_true	I_MC	Ошибка	ΔI
R = 0.75	100	39.00000000	40.17097044	1.17097044	3.90000000
R = 0.75	1000	39.00000000	41.31109061	2.31109061	1.23328829
R = 0.75	10000	39.00000000	38.21698524	0.78301476	0.39000000
R = 0.75	100000	39.00000000	39.04081370	0.04081370	0.12332883

Точка отсечки R = 0.95

Параметр	N	I_true	I_MC	Ошибка	ΔI
R = 0.95	100	39.00000000	38.74826359	0.25173641	3.90000000
R = 0.95	1000	39.00000000	48.21974528	9.21974528	1.23328829
R = 0.95	10000	39.00000000	37.30590435	1.69409565	0.39000000
R = 0.95	100000	39.00000000	38.94912269	0.05087731	0.12332883

Анализ результатов вычислений

По полученным результатам можно сделать следующие выводы.

Простое Монте-Карло демонстрирует обычную для этого метода сходимость: при увеличении числа выборок приближенное значение интеграла постепенно приближается к точному, а абсолютная ошибка уменьшается.

Метод Монте-Карло со стратификацией показал более высокую точность по сравнению с простым методом. Особенно удачным оказался вариант с шагом стратификации $h = 0.5$, для которого даже при относительно небольшом числе выборок ошибка получилась очень малой.

Метод выборки по значимости дал наилучший результат при использовании плотности $p(x) \sim x^2$. В этом случае приближенное значение интеграла во всех экспериментах совпало с точным значением. Это объясняется тем, что выбранная плотность совпадает по форме с подынтегральной функцией.

Метод многократной выборки по значимости также показал хорошую точность. Среди двух использованных схем весов немного лучше проявила себя схема balance, особенно при больших значениях N.

Метод русской рулетки оказался наименее устойчивым при малом числе выборок. Для небольших Nошибка могла быть достаточно большой, однако при увеличении числа выборок результаты также начали приближаться к точному значению.

Вывод

В ходе выполнения лабораторной работы были изучены базовые методы Монте-Карло, применяемые для численного интегрирования функций на заданном интервале. Для функции $f(x) = x^2$на отрезке $[2, 5]$было получено точное аналитическое значение интеграла, равное 39, и выполнено его численное вычисление несколькими методами Монте-Карло.

В работе были реализованы и исследованы следующие методы: простое Монте-Карло, Монте-Карло со стратификацией, выборка по значимости, многократная выборка по значимости и метод русской рулетки. Для каждого метода были рассчитаны приближенные значения интеграла при различных числах выборок, абсолютные ошибки и оценки погрешности. Это позволило сравнить методы между собой и оценить их точность и устойчивость.

По результатам экспериментов можно сделать вывод, что при увеличении числа выборок точность вычисления интеграла в целом возрастает, а ошибка уменьшается. Простое Монте-Карло показало ожидаемую сходимость, однако более совершенные модификации в ряде случаев дали более точные результаты. Особенно эффективными оказались метод стратификации и метод выборки по значимости. Наилучший результат был получен при использовании плотности $p(x) \sim x^2$, так как она согласована с видом подынтегральной функции, и в этом случае приближенное значение интеграла совпало с точным.

Метод многократной выборки по значимости также показал высокую точность и хорошую сходимость, особенно при использовании весовой схемы balance. Метод русской рулетки, напротив, продемонстрировал более высокий разброс результатов при малом числе выборок, но при больших значениях Nтакже приближался к точному значению интеграла.

Таким образом, цель лабораторной работы была достигнута: были изучены и практически реализованы основные методы Монте-Карло для вычисления интеграла, а также проведено сравнение их точности, особенностей и эффективности на конкретном примере.